

TP sur la commande prédictive: une étude de cas sur le système à quatre réservoirs

Yuqi LIU¹, Alexandre HACHE², Lucas GRUSS

1 Introduction & Préliminaires

1.1 Objectifs

La commande prédictive (MPC) est de plus en plus utilisée dans les secteurs industriels et financiers, grâce à des progrès relatifs à la puissance de calcul et aux méthodes de mise en oeuvre. Ce benchmark permet d'approfondir les questions de la formalisation du problème MPC et de sa mise en oeuvre: implémentation, réglage. Les objectifs de ce TP sont les suivants :

1. Questionner le lien entre le contrôle optimal et la commande prédictive.
2. Comprendre la mise en oeuvre des algorithmes de MPC dans l'environnement MATLAB.
3. Se familiariser avec les outils d'optimisation et les solveurs.
4. Maîtriser la configuration des méta-paramètres et des modèles pour la MPC.

1.2 Contexte

Le modèle de quadruple réservoir (QTP) est un système non linéaire multivariable, utilisé en recherche et dans les applications industrielles pour sa complexité et sa pertinence. Servant de référence pour tester des stratégies comme la Commande Prédictive (MPC pour Model Predictive Control), le QTP simule la dynamique des fluides de processus industriels avec quatre réservoirs interconnectés.

En tant que cas d'étude, le QTP, offre la possibilité d'une expérience pratique relative à la commande de systèmes multivariables (MIMO), possiblement non-linéaires et couplés.

Dans le secteur industriel, la problématique de contrôle du QTP trouve son parallèle dans des applications telles que le traitement chimique et l'assainissement de l'eau, pour lesquels le contrôle précis des niveaux et flux de fluides est crucial.

1.3 Environnement et support de travail

L'environnement suivant sera utilisé au cours du TP:

- Matlab (version $\geq 2020b$)
- L'expérience virtuelle: Exp1 - 4.

¹E-mail: yuqi.liu@imt-atlantique.fr

²E-mail: alexandre.hache@imt-atlantique.fr

- Supports de cours et de TP
- La toolbox CasADI. Elle est installée automatiquement par les codes qui vous sont fournis.

1.4 Compte-rendu

Au cours des trois sessions du TP, vous devrez répondre aux questions de ce document en vous aidant des codes fournis. Un rapport est attendu et devra être déposé par mail.

Les figures et les tableaux illisibles ne seront pas examinés.

Taille maximale des groupes : 2.

Format : PDF (Un rapport manuscrit et scanné est accepté).

Deadline : 21/04/2025

Soumettre à : yuqi.liu@imt-atlantique.fr

2 Processus et modélisation

Comme son nom l'indique, le processus considéré se compose de quatre réservoirs d'eau, de deux pompes et de deux vannes à trois voies. Chaque réservoir comporte un orifice par lequel le fluide s'écoule, du réservoir 3 au réservoir 1, du réservoir 4 au réservoir 2, puis des réservoirs 1 et 2 vers un bassin de rétention (voir figure 1). On cherche à concevoir des lois de commande capables de réguler le niveau d'eau dans les réservoirs 1 et 2 par le contrôle des pompes 1 et 2.

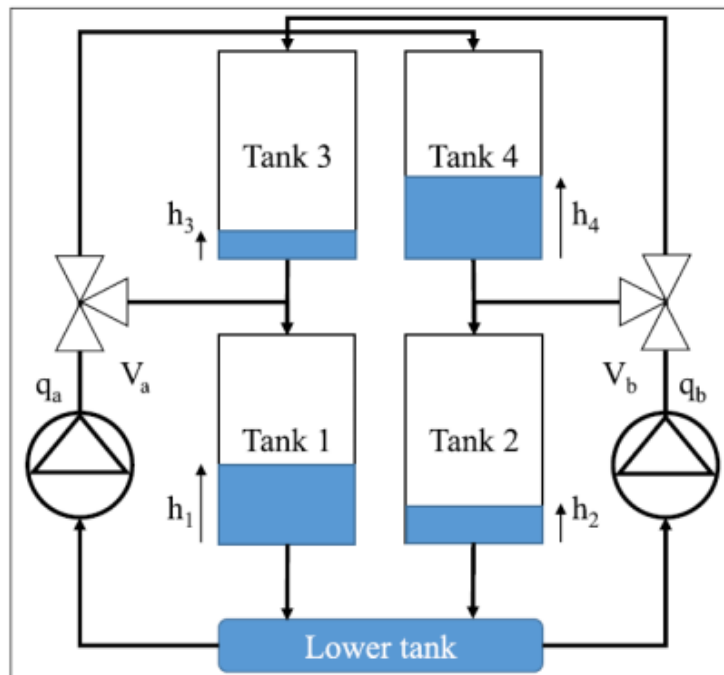


Figure 1: Quadruple tank Process

2.1 Modélisation

Chaque réservoir contient un volume de liquide $V = h \times S$ (m^3), où h est la hauteur du fluide et S est la section des réservoirs. L'évolution de la hauteur de fluide dans les réservoirs est régie par l'équation

suivante :

$$\dot{V} = q_{\text{in}} - q_{\text{out}},$$

où

- q_{in} est le débit d'entrée des pompes (m^3/s)
- q_{out} est le débit de sortie dû aux fuites (m^3/s)

La vitesse d'écoulement (en m/s) s'exprime comme $v = \sqrt{2gh}$, avec h la hauteur d'eau et g la gravité. Pour les débits d'entrée (pompe) et de sortie exprimés en (m^3/s), le principe de Torricelli, conduit pour l'évolution des quatre hauteurs d'eau à :

$$\begin{aligned}\frac{dh_1}{dt} &= -\frac{a_1\sqrt{2gh_1}}{S} + \frac{a_3\sqrt{2gh_3}}{S} + \frac{\gamma_a q_a}{S} \\ \frac{dh_2}{dt} &= -\frac{a_2\sqrt{2gh_2}}{S} + \frac{a_4\sqrt{2gh_4}}{S} + \frac{\gamma_b q_b}{S} \\ \frac{dh_3}{dt} &= -\frac{a_3\sqrt{2gh_3}}{S} + \frac{(1-\gamma_b)q_b}{S} \\ \frac{dh_4}{dt} &= -\frac{a_4\sqrt{2gh_4}}{S} + \frac{(1-\gamma_a)q_a}{S}\end{aligned}$$

Ici, a_i représente la section de fuite (m^2), q_i est le débit de la pompe, et γ_i est l'ouverture de la vanne i (comprise entre 0 et 1).

2.2 Paramètres du modèle

Table 1: Paramètres du modèle

Paramètre	Valeur	Unité	Description
a_1	1.31×10^{-4}	m^2	Surface de l'orifice de fuite
a_2	1.51×10^{-4}	m^2	Surface de l'orifice de fuite
a_3	9.27×10^{-5}	m^2	Surface de l'orifice de fuite
a_4	8.82×10^{-5}	m^2	Surface de l'orifice de fuite
γ_a	0.3	—	Ouverture de la vanne à trois voies
γ_b	0.4	—	Ouverture de la vanne à trois voies
S_c	0.06	m^2	Section transversale des réservoirs

Table 2: Contraintes sur les variables d'état

Variable	Min.	Max.	Unité	Description
h_1	0	1.36	m	État du système : niveau d'eau
h_2	0	1.36	m	État du système : niveau d'eau
h_3	0	1.36	m	État du système : niveau d'eau
h_4	0	1.36	m	État du système : niveau d'eau
q_a	0	3.26	m^3/h	Entrée du système : débit d'eau
q_b	0	4	m^3/h	Entrée du système : débit d'eau

Table 3: Condition initiale et référence de l'état

Variable	Valeur	Unité	Description
$h_{1,IC}$	0.01	m	Condition initiale
$h_{2,IC}$	1.2	m	Condition initiale
$h_{3,IC}$	0.65	m	Condition initiale
$h_{4,IC}$	0.65	m	Condition initiale
$h_{1,ref}$	0.8	m	Référence de commande
$h_{2,ref}$	0.6	m	Référence de commande

Pour un étude de sensibilité, on peut considérer différentes sources de perturbations ou d'incertitude.

Question 2.1.1 Énumérez les entrées de commande, les sorties à réguler et les états du système.

Question 2.1.2 Analysez au travers des équations le caractère linéaire/non-linéaire du système, le caractère couplé/non-couplé des variables d'état.

Question 2.1.3 Déterminez analytiquement la commande u_{ref} et les états x_{ref} en régime permanent.

Question 2.1.4 On cherche à réguler les niveaux des réservoirs 1 et 2. Pourquoi ne commandons-nous que 2 réservoirs au lieu de 4 ?

Question 2.1.5 Linéarisez le système autour du point d'équilibre de Question 2.1.3 par un développement de Taylor au premier ordre.

Question 2.1.6 Analysez la contrôlabilité du système linéarisé.

Question 2.1.7

2 fichiers sont présentés dans le dossier ./Exp1.QTP_OL.LQR/.

- Parameters.m
- TP_MCA.QTP.slx

Pour réaliser cette expérience, il faut exécuter d'abord le fichier './Parameters.m' puis le fichier 'TP_MCA.QTP.slx' dans la Simulink ou de command window.

Implémentez et simulez les systèmes linéarisé et non-linéaire dans 'TP_MCA.QTP.slx'. Confrontez les comportements.

Un bloc utile peut être trouvé dans la librairie : Simulink/Continuous/State-Space

Concevez un contrôleur LQ avec un modèle linéarisé pour compléter la simulation en boucle fermée.

2.3 Modélisation étendue

Ces questions explorent des défis typiques de l'ingénierie des systèmes : imperfections matérielles, incertitudes paramétriques et défaillances critiques. Vous mobiliserez vos connaissances en modélisation dynamique, linéarisation et analyse de robustesse pour évaluer l'impact de ces perturbations et proposer des correctifs.

Veillez fournir des solutions spécifiques de modélisation étendue à l'aide du modèle mathématique de QTP. Vous pouvez modifier le modèle QTP dans simulink ou l'équation dessus pour illustrer votre idée.

Question 2.2.1 Modélisez une imprécision des actionneurs où les débits q_a et q_b subissent une déviation aléatoire de $\pm 2\%$ autour de leur consigne. Intégrez cette incertitude dans les équations dynamiques.

Question 2.2.2 Évaluez l'impact d'une surestimation de 5% de a_1 (soit $a_1 = 1.38 \times 10^{-4} \text{ m}^2$) sur le niveau h_1 en régime permanent.

Question 2.2.3 Une fuite imprévue de $0.015 \text{ m}^3/\text{s}$ apparaît à $h_3 = 1.01 \text{ m}$. Proposer une structure de modèle qui puisse s'adapter à cette situation de fuite, afin de maintenir la stabilité de h_1 et h_2 .

3 Commande Prédictive Nonlinéaire avec CasADI

3.1 Principes

La commande prédictive non linéaire (NMPC) repose sur la résolution d'un problème d'optimisation dynamique où le modèle interne décrit explicitement les non-linéarités du système. Si le modèle nonlinéaire est incorporé directement dans l'algorithme MPC, le problème d'optimisation est alors un programmation non linéaire (NLP).

Voici la formulation d'une optimization

$$\begin{aligned} \min_w \quad & J(w) \\ \text{s.t.} \quad & G(w) = 0 \\ & H(w) \geq 0 \end{aligned}$$

où w est la variable de décision. $J(w)$ représente l'objectif, $G(w)$ est une contrainte d'égalité, $H(w)$ est une contrainte d'inégalité.

La question suivante permet de construire la formulation des NMPC étape par étape avec vous.

Question 3.1.1 Quel est l'objectif de contrôle de QTP? Proposez une fonction coût inspirée du LQ.

Question 3.1.2 Détailler la contrainte d'égalité et de non-égalité. Selon le modèle, limite d'entrée, limite d'état...

Question 3.1.3 Utiliser des couleurs différentes pour chaque variable et fonction dans l'optimisation. Marquez en rouge les variables qui sont renouvelées par le feedback en ligne à chaque pas de temps de contrôle. Les variables constantes sont marquées en bleu. Marquez en vert les variables qui dépendent des résultats de l'optimisation.

Question 3.1.4 Pour garantir la stabilité et la robustesse, l'ajout de $(u - u_{ref})^T R (u - u_{ref})$ à la fonction objective est une solution pratique (voir support). Quel est l'inconvénient d'utiliser le signal de commande de référence (e.g. résultat de Question 2.1.3) dans l'expression de la fonction coût ? Quelle alternative considérer ?

3.2 CasADi et mise en œuvre

La solution de modèle nonlinéaire de contrôle optimal, typiquement non convexe, nécessite des solveurs de NLP tels que Méthodes du Point Intérieur (IP) ou Programmation Quadratique Séquentielle (SQP). Les outils mathématiques symboliques fournissent un syntax général pour plusieurs types de solveurs. Ce syntax est plus proche de la formulation mathématique que du langage de programmation.

CasADi est une boîte à outils pour l'optimisation numérique et le contrôle optimal. En particulier, elle est utilisée pour formuler des problèmes NLP et interagir avec divers solveurs (fournis avec CasADi ou disponibles sous licence commerciale comme KNITRO ou Gurobi). Son principal avantage est qu'elle calcule toutes les dérivées du problème automatiquement et symboliquement. Cette approche est beaucoup plus performante que le calcul numérique avec des différences finies. Dans ce TP, vous construirez vos propres algorithmes NMPC avec le package CasADi.

Mise en œuvre

Étape 1 Ouvrez le fichier Exp2_QTP_NMPC/TP_MCA_QTP.slx dans Simulink. Double-cliquez sur le bloc MPC_controller. Cliquez sur 'source code'. L'algorithme MPC sera ouvert dans votre

fenêtre Matlab. Ce fichier est en fait './MPC_controller.m' dans le même répertoire que votre modèle Simulink.

Ce fichier est une classe de système matlab, une classe spéciale qui s'exécute à des pas de temps définis dans la simulation. Lisez la documentation de Matlab et le code existant pour comprendre la structure.

Étape 2 La précompilation du problème d'optimisation améliore l'efficacité des calculs MPC. Pour ce faire, un problème d'optimisation doit être construit avec certains paramètres inconnus. Attention, ces paramètres ne sont pas des paramètres de modèle, mais des variables à paramétrer en ligne par des mesures.

`function setupImpl(obj)` sera exécutée avant le début de la simulation. Elle est utilisée pour l'initialisation et la précompilation.

`function u = stepImpl(obj)` sera exécutée pendant la simulation. Le pas de temps est défini dans `function sts = getSampleTimeImpl(obj)`. Nous procédons ici à l'optimisation de la solution.

Les paramètres du système et du MPC sont définis dans le fichier `Parameters.m`. Exécutez ce script pour les charger dans le workspace.

Étape 3 Construisez votre propre problème de NMPC. CasADI syntax est bien présenté sur son site web. Voici quelques conseils.

L'option d'optimisation pourrait être

```
opti = casadi.Opti();
opti.solver('ipopt',...
    struct('print_time', false), ...
    struct('print_level', 0, 'max_iter', 100));
```

Lors de la définition des variables, vous utiliserez deux commandes ci-dessous.

`opti.variable()` %instancie la variable de décision.

`opti.parameter()` %instancie la variable paramètre.

Pour ajouter une contrainte à l'optimisation, utilisez `opti.subject_to()`.

Pour définir une fonction objectif, utilisez `opti.minimize()`.

Nous utilisons la mémoire locale de la classe matlab. C'est-à-dire que toutes les variables commençant par `obj` seront mémorisées pendant toute la durée de la simulation.

Les suggestions pour chaque ligne sont données dans `MPC_controller.m`. Remplacez le commentaire avec `**` par vos codes.

Étape 4 Attribuer une valeur à l'optimisation en ligne. Au cours de la simulation, vous devez attribuer un retour d'information à l'optimisation. Utilisez la fonction `opti.set_value`.

Si tout est bien défini, résolvez l'optimisation avec `solution = opti.solve()` Extrayez vos résultats à l'aide de `u_res = sol.value(u)`

Question 3.2.1 Exécutez l'implémentation NMPC en ouvrant et en exécutant le modèle Simulink `TP_MCA_QTP.slx`. Analysez les résultats produits: respectent-ils votre cahier des charges? Sinon, quelles sont vos propositions ?

Question 3.2.2 En vous penchant sur le script `Parameters.m`, pouvez-vous dire si quelque chose cloche ? Indice: regardez les valeurs des signaux de commande prises lors de la simulation. Proposez un réglage de la loi de commande. Pour cela, vous pouvez ajuster les paramètres suivants:

- horizon de prédiction
- pas de temps de discrétisation
- matrices de pondération
- introduction d'un coup terminal

Question 3.2.3 Comparez et commentez les résultats de simulation de la MPC lorsque tout l'état doit suivre la référence x_{ref} ou quand seule les sorties doivent suivre y_{ref} (changez la valeur de Q).

Question 3.2.4 Argumentez votre choix, sur la base du compromis à réaliser entre performance et coût de calcul.

Question 3.2.5 Comparez et commentez les résultats de simulation en boucle fermée avec la MPC, le LQ, et la commande en boucle ouverte. Une fonction magique et utile dans Simulink. En haut DEBUG → triangle sous 'performance' → Simulink Profiler. Il vous donne des informations sur le calcul.

4 Commande Prédictive Linéaire

Dans les applications pratiques, les exigences informatiques de l'optimisation limitent l'échantillonnage réalisable de MPC. Les modèles linéaires, avec leur efficacité de calcul inhérente, représentent une alternative convaincante pour la mise en œuvre en temps réel. Ce chapitre explore l'équilibre entre la vitesse de calcul offerte par les approximations linéaires et leur capacité à maintenir la précision du contrôle dans des conditions dynamiques.

4.1 Linéarisation

Pour un système dynamique non linéaire

$$\dot{x} = f(x, u)$$

La mesure (x_t, u_t) est utilisée comme point de fonctionnement pour la linéarisation. On procède alors à Nous choisissons une expansion de Taylor du premier ordre pour l'approximer.

$$\dot{x} = f(x_t, u_t) + \frac{\partial f}{\partial x}|_{x_t, u_t}(x - x_t) + \frac{\partial f}{\partial u}|_{x_t, u_t}(u - u_t)$$

Question 4.1.1 Quel est le modèle linéarisé autour du point de référence de la question 2.1.3 ?

Question 4.1.2 Quel est le modèle linéarisé en un point arbitraire (x_t, u_t) ?

Question 4.1.3 Quelle est la différence entre le MPC linéaire et le LQR ?

Mise en œuvre A

Étape 1 Ouvrez le fichier `Exp3_QTP_LMPC/Parameters.m`. Définissez votre modèle de question 4.1.1 et nommez-le `f_lref`.

Il faut définir le modèle avec `casadi.Function()`. Voici un exemple.

```
f_lref = casadi.Function('ode', {x, u}, {x+u+x*u}, ...  
    {'state', 'input'}, {'dx'});
```

où l'argument 'ode' est le nom de cette fonction. $\{x, u\}$ sont les entrées du modèle. $\{x+u+x*u\}$ est un exemple fictif du modèle ODE. $\{'state', 'input'\}$ sont les noms des entrées. $\{'dx'\}$ est le nom du modèle ODE.

Étape 2 Remplissez le fichier `MPC_controller.m` avec votre algorithme MPC.

Étant donné que le modèle est linéaire, nous pouvons utiliser un solveur de programmation quadratique `qpooases`. Voici la configuration recommandée.

```
opti = casadi.Opti('conic');  
opts = struct();  
opts.print_time = false;  
opts.printLevel = 'none';
```

```
opti.solver('qpoads',opts);
```

Mise en œuvre B

Étape 1 Ouvrez le fichier Exp4_QTP_AMPC/Parameters.m. Définissez votre modèle de question 4.1.2. Linéarisez votre modèle en fonction de l'évolution des moyennes à chaque pas de temps. Incorporez cette fonction à votre modèle pour le rendre 'adaptatif' à l'état en temps réel. Nommez-le `f_lact`. La différenciation partielle est donnée.

Étape 2 Remplissez le fichier MPC_controller.m avec votre algorithme MPC.

Exécutez TP_MCA_QTP.slx et répondre aux questions suivantes.

Question 4.2.1 Comparez la précision et la vitesse de convergence entre les MPC linéaires et non linéaires. Évaluez les raisons de ce résultat.

Question 4.2.2 Comparez du temps de calcul de la MPC linéaire et de la MPC non linéaire. Ajuster les hyperparamètres, tels que l'horizon, afin de trouver un compromis entre les performances et le coût en temps des deux méthodes.

Question 4.2.3 Résumer les résultats précédents, déterminer des scénarios et des objets pour l'application de MPC linéaire et non-linéaire.